ARIN5204: Reinforcement Learning

2.3 Value Function Approximation

Dr. Long Chen

longchen@ust.hk
Assistant Professor, Dept. of CSE
The Hong Kong University of Science and Technology (HKUST)

March 23, 2026

Some slides are from: Katerina Fragkiadaki (CMU), Davild Silver (DeepMind), Hado van Hasselt (DeepMind).

Function Approximation and Deep RL

- The policy, value function, model, and agent state update are all functions
- We want to learn these from experience
- If there are too many states, we need to approximate
- Real-world problems with a **large** number of states, *e.g.*,
  - Backgammon: 10^{20} states
  - Go: 10^{170} states
  - Helicopter: continuous state space
  - Robots: real world

Value Function Approximation (VFA)

- So far, we have represented the value function by a lookup table
  - Every state s has an entry $V(s)$ or
  - Every state-action pair (s, a) has an entry $Q(s, a)$
- Problem with large MDPs:
  - There are too many states and/or actions to store in memory
  - It is too slow to learn the value of each state individually
- Tabular methods that enumerate every single state do not work

Which Function Approximation?

There are many function approximators, *e.g.*,

- Linear combinations of features
- Neural networks
- Decision tree
- Nearest neighbour
- Fourier/wavelet bases
- ...

Classes of Function Approximation

- Q: Which function approximation should you choose?
- This depends on your goals:
  - Tabular: good theory but does not scale/generalize
  - Linear: reasonably good theory, but requires good features
  - Non-linear: less well-understood, but scales well
  - Flexible, and less reliant on picking good features first (*e.g.*, by hand)
  - (Deep) neural nets often perform quite well, and remain a popular choice
    - This is often called deep reinforcement learning, when using (deep) neural network to represent these functions

Large-Scale Reinforcement Learning

- Tabular: a table with an entry for each MDP state
- How can we scale up the model-free methods for prediction and control from the last two lectures?
- Linear function approximation
  - Consider fixed agent state update (*e.g.*, $s_t = o_t$)
  - Fixed feature map: $\boldsymbol{x} : s \to \mathbb{R}^n$
  - Values are linear function of features $V(s; \boldsymbol{w}) = \boldsymbol{w}^T \boldsymbol{x}(s)$

Classes of Functions Approximation

- Solution for large MDPs
  - Estimate value function with function approximation

$$V(s; \boldsymbol{w}) \approx V_\pi(s) \quad \text{or} \quad Q(s, a; \boldsymbol{w}) \approx Q_\pi(s, a)$$

  - Generalize from seen states to unseen states
  - Update parameters $\boldsymbol{w}$ using MC or TD learning
- Differentiable function approximation
  - $V(s; \boldsymbol{w})$ is a differentiable function of $\boldsymbol{w}$, could be non-linear
  - *e.g.*, a convolutional neural network that takes pixel as input
  - Another interpretation: features are not fixed, but learnt

Function Approximation for RL Problems

In principle, any function approximator can be used, but RL has specific properties,

- Experience is not *i.i.d.*, *i.e.*, successive time steps are correlated
- Agent's policy affects the data it receives
- Regression targets can be non-stationary
  - ... because of changing policies (which can change the target and the data!)
  - ... because of bootstrapping
  - ... because of non-stationary dynamics (*e.g.*, other learning agents)
  - ... because the world is large (never quite in the same state)

Agent State Update

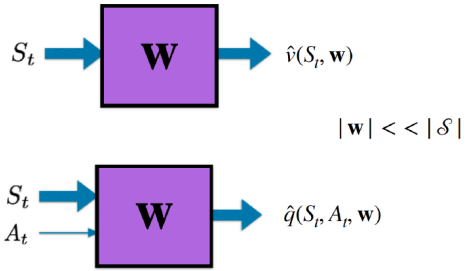
- When the environment state is not fully observable ($s_t^{env} \neq o_t$)
- Use the agent state (with parameters $\boldsymbol{w}$)

$$s_t = U(s_{t-1}, a_{t-1}, o_t; \boldsymbol{w})$$

- Henceforth, s_t denotes the agent state
- Think of this as either a vector inside the agent, or, in the simplest case, just the current observation: $s_t = o_t$

Value Function Approximation (VFA)

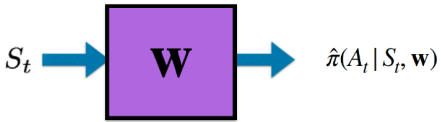
- Value function approximation (VFA) replaces the table with a general parameterized form:



- When we update the parameters $\boldsymbol{w}$, the values of many states change simultaneously!

Policy Approximation

Policy approximation replaces the table with a general parameterized form



Function Approximation Examples

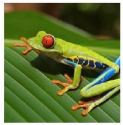
Image representation for classification

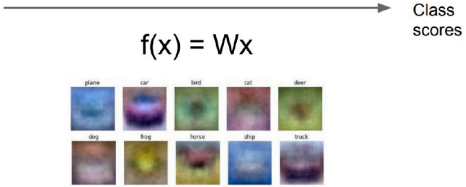
(assume given a set of labels)
{dog, cat, truck, plane, ...}

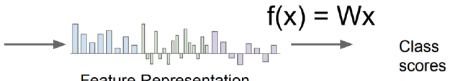
cat
dog
bird
deer
truck

Function Approximation Examples

Pixel space

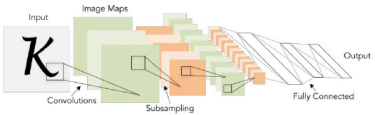


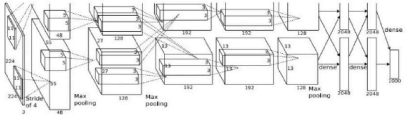


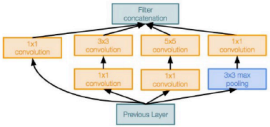


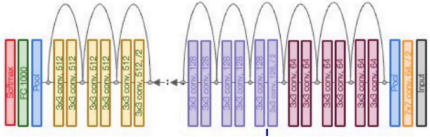
Function Approximation Examples

Convolutional neural network (CNN) architectures



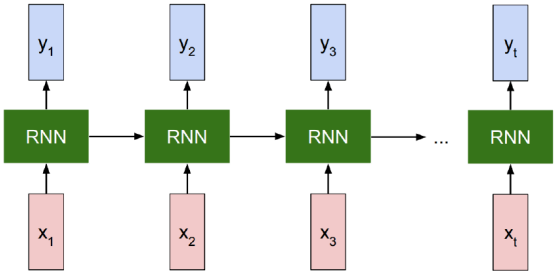






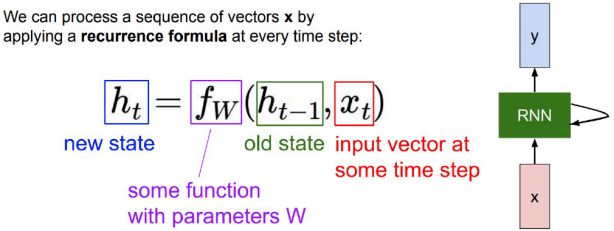
Function Approximation Examples

Recurrent neural network (RNN) architectures



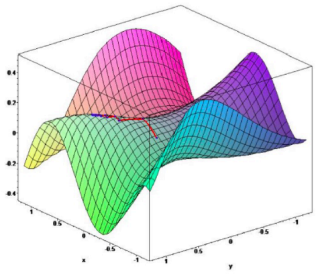
Function Approximation Examples

Recurrent neural network (RNN) architectures



Background: Gradient Descent

Let $J(\boldsymbol{w})$ be a differentiable function of parameter vector $\boldsymbol{w}$



- Define the gradient of $J(\boldsymbol{w})$ to be:

$$\nabla_{\boldsymbol{w}} J(\boldsymbol{w}) = \begin{pmatrix} \frac{\partial J(\boldsymbol{w})}{\partial w_1} \\ \vdots \\ \frac{\partial J(\boldsymbol{w})}{\partial w_n} \end{pmatrix}$$

- To find a local minimum of $J(\boldsymbol{w})$, adjust $\boldsymbol{w}$ in direction of the negative gradient

$$\Delta \boldsymbol{w} = -\frac{1}{2}\alpha \nabla_{\boldsymbol{w}} J(\boldsymbol{w})$$

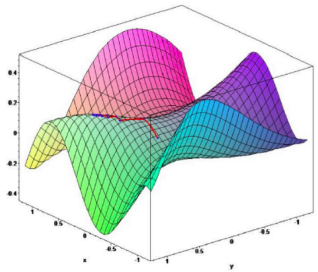
where α is a step-size parameter

Background: Gradient Descent

Let $J(\boldsymbol{w})$ be a differentiable function of parameter vector $\boldsymbol{w}$

- Define the gradient of $J(\boldsymbol{w})$ to be:

$$\nabla_{\boldsymbol{w}} J(\boldsymbol{w}) = \begin{pmatrix} \frac{\partial J(\boldsymbol{w})}{\partial w_1} \\ \vdots \\ \frac{\partial J(\boldsymbol{w})}{\partial w_n} \end{pmatrix}$$



- Starting from a guess $\boldsymbol{w}_0$
- We consider the sequence $\boldsymbol{w}_0, \boldsymbol{w}_1, \boldsymbol{w}_2, ...$
- *s.t.*, $\Delta \boldsymbol{w}_{n+1} = -\frac{1}{2}\alpha \nabla_{\boldsymbol{w}} J(\boldsymbol{w}_n)$
- We then have $J(\boldsymbol{w}_0) \geq J(\boldsymbol{w}_1) \geq (\boldsymbol{w}_2) \geq ...$

Value Function Approx. by Stochastic Gradient Descent

- Goal: find parameter vector $\boldsymbol{w}$ minimizing mean-squared error between the true value function $V_\pi(s)$ and its approximation $V(s;\boldsymbol{w})$

$$J(\boldsymbol{w}) = \mathbb{E}_{s\sim d}[(V_\pi(s) - V(s;\boldsymbol{w}))^2]$$

  Where d is a distribution over states (induced by policy and dynamics)
- Gradient descent finds a local minimum

$$\Delta w = -\frac{1}{2}\alpha\nabla_{\boldsymbol{w}} J(\boldsymbol{w}) = \alpha\mathbb{E}_d\left[(V_\pi(s) - V(s;\boldsymbol{w}))\nabla_{\boldsymbol{w}} V(s;\boldsymbol{w})\right]$$

- Stochastic gradient descent (SGD), samples the gradient

$$\Delta\boldsymbol{w} = \alpha\left(G_t - V(s_t;\boldsymbol{w})\right)\nabla_{\boldsymbol{w}} V(s_t;\boldsymbol{w})$$

- Note: Monte Carlo return G_t is a sample for $V_\pi(s_t)$
- Expected update is equal to full gradient update
- We often write $\nabla V(s_t)$ as short hand for $\nabla_{\boldsymbol{w}} V(s_t;\boldsymbol{w})|_{\boldsymbol{w}=\boldsymbol{w}_t}$

Feature Vectors

- Represent state by a **feature vector**

$$\boldsymbol{x}(s) = \begin{pmatrix} x_1(s) \\ x_2(s) \\ \vdots \\ x_n(s) \end{pmatrix}$$

- $\boldsymbol{x} : s \to \mathbb{R}^n$ is a fixed mapping from state (*e.g.*, observation) to features
- Short hand: $\boldsymbol{x}_t = \boldsymbol{x}(s_t)$

Linear Value Function Approximation

- Represent the value function by a linear combination of features

$$V(s; \boldsymbol{w}) = \boldsymbol{x}(s)^T \boldsymbol{w} = \sum_{j=1}^{n} \boldsymbol{x}_j(s) \boldsymbol{w}_j$$

- Objective function is quadratic in parameters bw

$$J(\boldsymbol{w}) = \mathbb{E}_{s \sim d}\left[\left(V_\pi(s) - \boldsymbol{x}(s)^T \boldsymbol{w}\right)^2\right]$$

- Stochastic Gradient Descent: The update rule is particularly simple

$$\nabla_{\boldsymbol{w}} V(s_t, \boldsymbol{w}) = \boldsymbol{x}(s_t) = \boldsymbol{x}_t$$

$$\Delta \boldsymbol{w} = \alpha \left(V_\pi(s_t) - V(s_t; \boldsymbol{w})\right) \boldsymbol{x}_t$$

- **Update = step-size × prediction error × feature value**

Incremental Prediction Algorithm

- Have assumed true value function $V_\pi(s)$ given by supervisor
- But in RL there is no supervisor, only rewards
- In practice, we substitute a target for $V_\pi(s)$
  - For MC, the target is the return G_t

$$\Delta \boldsymbol{w} = \alpha \left(G_t - V(s_t; \boldsymbol{w})\right) \nabla_{\boldsymbol{w}} V(s_t; \boldsymbol{w})$$

  - For TD(0), the target is the TD target

$$\Delta \boldsymbol{w} = \alpha \left(r_t + \gamma V(s_{t+1}; \boldsymbol{w}) - V(s_t; \boldsymbol{w})\right) \nabla_{\boldsymbol{w}} V(s_t; \boldsymbol{w})$$

  - For TD(λ), the target is the λ-return G_t^λ

$$\Delta \boldsymbol{w} = \alpha \left(G_t^\lambda - V(s_t; \boldsymbol{w})\right) \nabla_{\boldsymbol{w}} V(s_t; \boldsymbol{w})$$

$$G_t^\lambda = r_t + \gamma \cdot \left((1-\lambda) V(s_{t+1}) + \lambda G_{t+1}^\lambda\right)$$

Monte Carlo with Value Function Approximation

- The return G_t is an unbiased, noisy sample of true value $V_\pi(s_t)$
- Can therefore apply supervised learning to "training data":

$$< s_1, G_1 >, < s_2, G_2 >, ..., < s_T, G_T >$$

- For example, using linear Monte-Carlo policy evaluation

$$\Delta \boldsymbol{w} = \alpha \left(G_t - V(s_t; \boldsymbol{w}) \right) \nabla_{\boldsymbol{w}} V(s_t; \boldsymbol{w}) = \alpha \left(G_t - V(s_t; \boldsymbol{w}) \right) \boldsymbol{x}_t$$

- Linear Monte-Carlo evaluation converges to a local optimum
- Even when using non-linear value function approximation it converges (but perhaps to a local optimum)

Monte Carlo with Value Function Approximation

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ using π
  Loop for each step of episode, $t = 0, 1, \ldots, T-1$:
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[G_t - \hat{v}(S_t,\mathbf{w})\big]\nabla\hat{v}(S_t,\mathbf{w})$

TD Learning with Value Function Approximation

- The TD-target $r_{t+1} + \gamma V(s_{t+1}; \boldsymbol{w})$ is a biased sample of true value $V_\pi(s_t)$
- Can still apply supervised learning to "training data"

$$< s_1, r_1 + \gamma V(s_1; \boldsymbol{w}) >, < s_2, r_2 + \gamma V(s_2; \boldsymbol{w}) >, ..., < s_T, r_T + \gamma V(s_T; \boldsymbol{w}) >$$

- For example, using linear TD(0)

$$\Delta \boldsymbol{w} = \alpha \left(r_t + \gamma V(s_{t+1}; \boldsymbol{w}) - V(s_t; \boldsymbol{w}) \right) \nabla_{\boldsymbol{w}} V(s_t; \boldsymbol{w}) = \alpha \cdot \delta_t \cdot \boldsymbol{x}_t$$

  where $\delta_t = r_t + \gamma V(s_{t+1}; \boldsymbol{w}) - V(s_t; \boldsymbol{w})$ is "TD error"
- This is akin to non-stationary regression problem
- But its a bit different: the target depends on our parameters!
- **We ignore the dependence of the target on $\boldsymbol{w}$! We call it the semi-gradient method!**

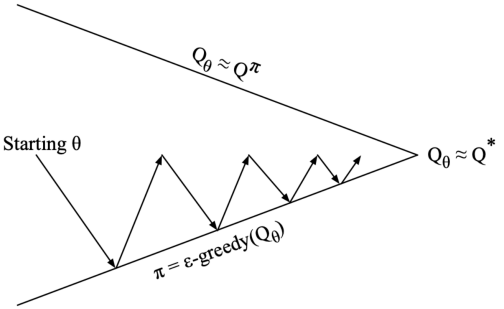
TD Learning with Value Function Approximation

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \to \mathbb{R}$ such that $\hat{v}(\text{terminal},\cdot) = 0$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose $A \sim \pi(\cdot|S)$
    Take action A, observe R, S'
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R + \gamma\hat{v}(S',\mathbf{w}) - \hat{v}(S,\mathbf{w})\big]\nabla\hat{v}(S,\mathbf{w})$
    $S \leftarrow S'$
  until S is terminal

Control with Value Function Approximation



- **Policy evaluation**: Approximate policy evaluation, $Q(s_t, a_t; \boldsymbol{w}) \approx Q_\pi$
- **Policy improvement**: ϵ-greedy policy improvement

Action-Value Function Approximation

- Should we use action-in, or action-out?
  - Action in: $Q(s, a; \boldsymbol{w}) = \boldsymbol{w}^T \boldsymbol{x}(s, a)$
  - Action out: $Q(s; \boldsymbol{w}) = \boldsymbol{W}\boldsymbol{x}(s)$ such that $Q(s, a; \boldsymbol{w}) = Q(s; \boldsymbol{w})[a]$
- Unclear which is better in general
- If we want to use continuous actions, action-in is easier (later lecture)
- For (small) discrete action spaces, action-out is common (*e.g.*, DQN)

Convergence Questions

- When do incremental prediction algorithms converge?
  - When using bootstrapping (*i.e.*, TD)?
  - When using (*e.g.*, linear) value function approximation?
  - When using off-policy learning?
- Ideally, we would like algorithms that converge in all cases
- Alternatively, **we want to understand when algorithms do, or do not, converge**

Convergence of MC

- With linear functions (and suitably decaying step size), MC converges to

$$\boldsymbol{w}_{MC} = \arg\min_{\boldsymbol{w}} \mathbb{E}_\pi[(G_t - V_{\boldsymbol{w}}(s_t))^2] = \mathbb{E}_\pi[\boldsymbol{x}_t \boldsymbol{x}_t^T]^{-1} \mathbb{E}_\pi[G_t \boldsymbol{x}_t]$$

  (Notation: here the state distribution implicitly depends on π)
- Verifying the fixed point:

$$\begin{aligned}
\nabla_{\boldsymbol{w}_{MC}} \mathbb{E}[(G_t - V_{\boldsymbol{w}}(s_t))^2] = \mathbb{E}[(G_t - V_{\boldsymbol{w}_{MC}}(s_t))\boldsymbol{x}_t] &= 0 \\
\mathbb{E}[(G_t - \boldsymbol{x}_t^T \boldsymbol{w}_{MC})\boldsymbol{x}_t] &= 0 \\
\mathbb{E}[G_t \boldsymbol{x}_t - \boldsymbol{x}_t \boldsymbol{x}_t^T \boldsymbol{w}_{MC}] &= 0 \\
\mathbb{E}[\boldsymbol{x}_t \boldsymbol{x}_t^T] \boldsymbol{w}_{MC} &= \mathbb{E}[G_t \boldsymbol{x}_t] \\
\boldsymbol{w}_{MC} &= \mathbb{E}[\boldsymbol{x}_t \boldsymbol{x}_t^T]^{-1} \mathbb{E}[G_t \boldsymbol{x}_t]
\end{aligned}$$

- Agent state s_t does not have to be Markov:
  the fixed point only depends on observed data (and features)

Convergence of TD

- With linear functions, TD converges to

$$\boldsymbol{w}_{TD} = \mathbb{E}[\boldsymbol{x}_t(\boldsymbol{x}_t - \gamma \boldsymbol{x}_{t+1})^T]^{-1}\mathbb{E}[r_t \boldsymbol{x}_t]$$

  (in continuing problems with fixed $\gamma < 1$, and decaying $\alpha_t \to 0$)
- Verify (assuming α_t does not correlate with $r_{t+1}, \boldsymbol{x}_t, \boldsymbol{x}_{t+1}$

$$\begin{aligned}
\mathbb{E}[\Delta \boldsymbol{w}_{TD}] = 0 &= \mathbb{E}[\alpha_t(r_t + \gamma \boldsymbol{x}_{t+1}^T \boldsymbol{w}_{TD} - \boldsymbol{x}_t^T \boldsymbol{w}_{TD})\boldsymbol{x}_t] \\
0 &= \mathbb{E}[\alpha_t r_t \boldsymbol{x}_t] + \mathbb{E}[\alpha_t \boldsymbol{x}_t(\gamma \boldsymbol{x}_{t+1}^T - \boldsymbol{x}_t^T)\boldsymbol{w}_{TD}] \\
\mathbb{E}[\alpha_t \boldsymbol{x}_t(\boldsymbol{x}_t^T - \gamma \boldsymbol{x}_{t+1}^T)]\boldsymbol{w}_{TD} &= \mathbb{E}[\alpha_t r_t \boldsymbol{x}_t] \\
\boldsymbol{w}_{TD} &= \mathbb{E}[\boldsymbol{x}_t(\boldsymbol{x}_t^T - \gamma \boldsymbol{x}_{t+1}^T)]^{-1}\mathbb{E}[\alpha_t r_t \boldsymbol{x}_t]
\end{aligned}$$

- This differs from the MC solution
- Typically, the asymptotic MC solution is preferred (smallest prediction error)
- TD often converges faster (especially intermediate $\lambda \in [0,1]$ or $n \in 1, ..., \infty$

Convergence of TD

- With linear functions, TD converges to

$$\boldsymbol{w}_{TD} = \mathbb{E}[\boldsymbol{x}_t(\boldsymbol{x}_t^T - \gamma \boldsymbol{x}_{t+1}^T)]^{-1}\mathbb{E}[\alpha_t r_t \boldsymbol{x}_t]$$

- Let $\overline{VE}(\boldsymbol{w})$ denote the value error:

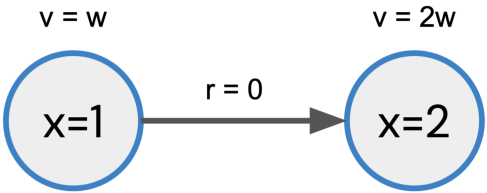
$$\overline{VE}(\boldsymbol{w}) = ||V_\pi - V_{\boldsymbol{w}}||_{d_\pi} = \sum_{s\sim S} d_\pi(s)\left(V_\pi(s) - V_{\boldsymbol{w}}(s)\right)^2$$

- The Monte Carlo solution minimizes the value error

Theorem

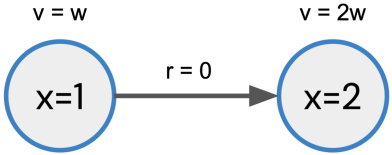
$$\overline{VE}(\boldsymbol{w}_{TD}) \leq \frac{1}{1-\gamma}\overline{VE}(\boldsymbol{w}_{MC}) = \frac{1}{1-\gamma}\min_{\boldsymbol{w}}\overline{VE}(\boldsymbol{w})$$

Example of Divergence



Q: What if we use TD only on this transition?

Example of Divergence

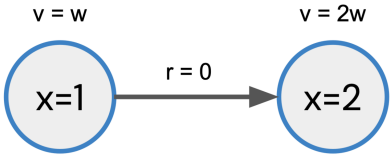


Q: What if we use TD only on this transition?

$$\begin{aligned} w_{t+1} &= w_t + \alpha_t(r + \gamma V(s') - V(s))\nabla V(s) \\ &= w_t + \alpha_t(r + \gamma V(s') - V(s))x(s) \\ &= w_t + \alpha_t(0 + \gamma \cdot 2w_t - w_t) \\ &= w_t + \alpha_t(2\gamma - 1)w_t \end{aligned}$$

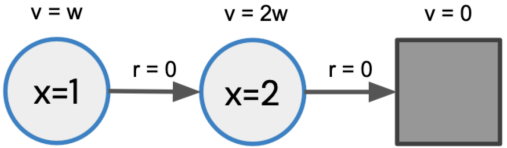
Consider $w_t > 0$. If $\gamma > \frac{1}{2}$, then $w_{t+1} > w_t$. $\Rightarrow \lim_{t\to\infty} w_t = \infty$

Example of Divergence



- Algorithms that combine
  - Bootstrapping
  - Off-policy learning, and
  - Function approximation
  - ... may diverge
- This is sometimes called the **deadly triad**.

Deadly Triad: Off-Policy

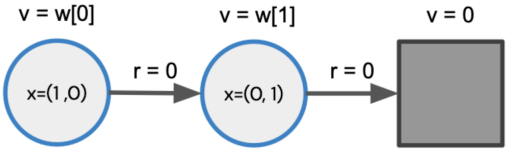


- Consider sampling on-policy, over an episode. Update:

$$\Delta w = \alpha(0 + \gamma \cdot 2w - w) + \alpha(0 + \gamma 0 - 2w) \tag{1.1}$$
$$= \alpha(2\gamma - 3)w \tag{1.2}$$

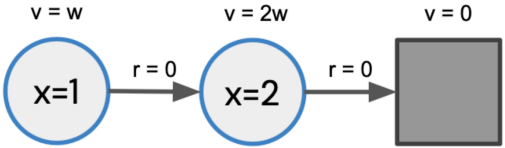
- This multiplier is negative, for all $\gamma \in [0, 1]$
- $\Rightarrow$ convergence (w goes to zero, which is optimal here)

Deadly Triad: Function Approximation



- With tabular feature, this is just regression
- Answer may be sub-optimal, but no divergence occurs
- Specifically, if we only update $V(s)$ (=left-most state):
  - $V(s) = w[0]$ will converge to $\gamma V(s')$
  - $V(s') = w[1]$ will stay where it was initialized

Deadly Triad: Bootstrapping



- What if we use multiple-step returns?
- Still consider only updating the left-most state

$$\begin{aligned}\Delta w &= \alpha\left(r+\gamma(G_t^\lambda - V(s))\right)\\ &= \alpha(r+\lambda(1-\lambda)V(s')+\lambda(r'+V(s''))-V(s)) \quad r=r'=V(s'')=0\\ &= \alpha(2\gamma(1-\lambda)-1)w\end{aligned}$$

- The multiplier is negative when $2\gamma(1-\lambda)<1 \Rightarrow \lambda > 1-\frac{1}{2\gamma}$
- *e.g.*, where $\gamma = 0.9$, then we need $\lambda > \frac{4}{9} \approx 0.45$

Convergence of Prediction and Control Algorithms

On/Off-Policy	Algorithm	Table Lookup	Linear	Non-Linear
On-Policy	MC	✓	✓	✓
	TD	✓	✓	×
Off-Policy	MC	✓	✓	✓
	TD	✓	×	×

- Tabular control learning algorithms (*e.g.*, Q-learning) can be extended to FA (*e.g.*, Deep Q Network DQN)
- The theory of control with function approximation is not fully developed
- Tracking is often preferred to convergence (*i.e.*, continually adapting the policy instead of converging to a fixed policy)

Deep Reinforcement Learning

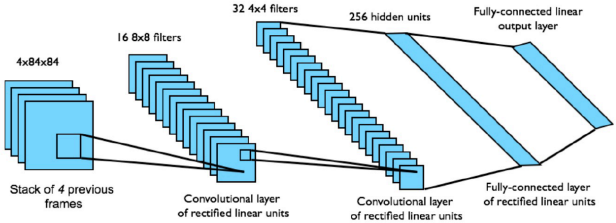
DL: Deep Learning; RL: Reinforcement Learning

- DL: It requires large amounts of hand-labelled training data.
- RL: It can learn from a scalar reward signal that is frequently sparse, noisy, and delayed.
- DL: It assumes the data samples to be independent.
- RL: It typically encounters sequences of highly correlated states.
- DL: It assumes a fixed underlying distribution.
- RL: The data distribution changes as the algorithm learns new behaviors.

Playing Atari with Deep Reinforcement Learning. In NIPS workshop, 2013.

DQN in Atari

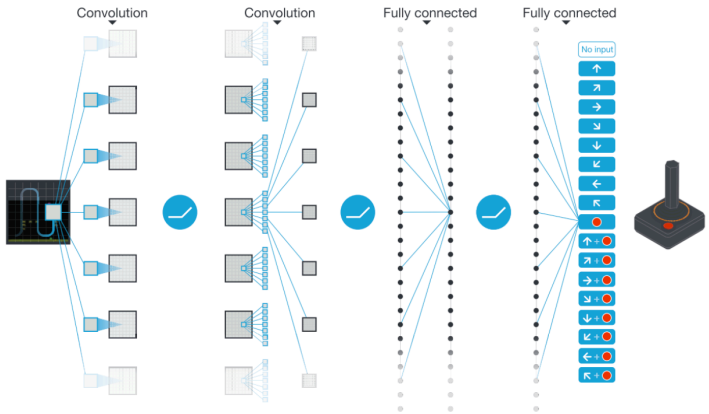
- End-to-end learning of values $Q(s, a)$ from pixels s
- Input state s is a stack of raw pixels from the last 4 frames
- Output is $Q(s, a)$ for 18 joystick/button positions
- Reward is the *change in score* for the step



Network architecture and hyperparameters fixed across all games

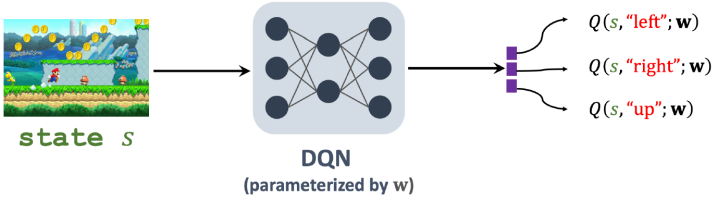
DQN

Approximate the optimal action-value function $Q_\pi(s,a)$ by $Q(s,a;\boldsymbol{w})$

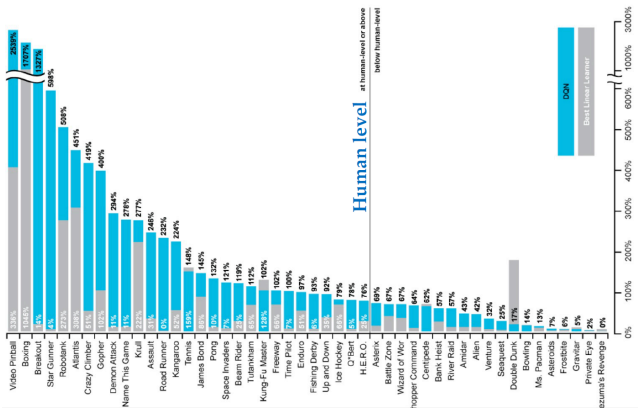


DQN

Approximate the optimal action-value function $Q_\pi(s, a)$ by $Q(s, a; \boldsymbol{w})$



DQN Results in Atari



$$100 \times \frac{\text{DQN scorerandom play score}}{\text{human scorerandom play score}}$$

Temporal Difference (TD) Learning

- Observe state s_t and perform action a_t
- Environment provides new state s_{t+1} and reward r_t
- TD target: $y_t = r_t + \gamma \cdot \max_a Q(s_{t+1}, a; \boldsymbol{w})$
- TD error: $\delta_t = Q_t - y_t$, where $Q_t = Q(s_t, a_t; \boldsymbol{w})$
- Goal: Make Q_t close to y_t, for all t. (Equivalently, make δ_t^2 small)
- **Q-Learning**: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$

Temporal Difference (TD) Learning

- TD error: $\delta_t = Q_t - y_t$, where $Q_t = Q(s_t, a_t; \boldsymbol{w})$
- TD learning: Find $\boldsymbol{w}$ by minimizing $L(\boldsymbol{w}) = \frac{1}{T}\sum_{t=1}^{T} \frac{\delta_t^2}{2}$
- Online gradient descent:
  - Observe (s_t, a_t, r_t, s_{t+1}) and compute δ_t
  - Compute gradient $\boldsymbol{g}_t = \frac{\partial \delta_t^2/2}{\partial \boldsymbol{w}} = \delta_t \cdot \frac{\partial Q(s_t, a_t; \boldsymbol{w})}{\partial \boldsymbol{w}}$
  - Gradient descent: $\boldsymbol{w} \leftarrow \boldsymbol{w} - \alpha \cdot \boldsymbol{g}_t$
  - Discard (s_t, a_t, r_t, s_{t+1}) after using it

Shortcoming 1: Waste of Experience

- A transition: (s_t, a_t, r_t, s_{t+1})
- Experience: all the transitions, for $t = 1, 2, \ldots$
- Previously, we discard (s_t, a_t, r_t, s_{t+1}) after using it
- It is a waste.

Shortcoming 2: Correlated Updates

- Previously, we use (s_t, a_t, r_t, s_{t+1}) sequentially, for $t = 1, 2, ...$ to update $\boldsymbol{w}$.
- Consecutive states, s_t and s_{t+1}, are strongly correlated (which is bad).
- It violates a commonly held assumption for stochastic gradient (similar issue as continual learning)

Extra Reading Materials

- Playing Atari with Deep Reinforcement Learning. In NIPS workshop, 2013.
- Human-level Control through Deep Reinforcement Learning. Nature, 2015.